Inteligencia Artificial

Episodio X (Parte 2)
Sistemas Basados en Conocimiento:
DeLP
Programación Lógica Rebatible

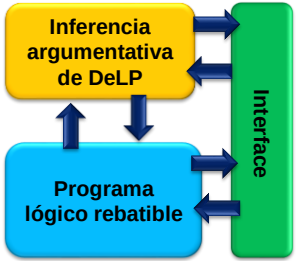


Dr. ALEJANDRO JAVIER GARCÍA http://cs.uns.edu.ar/~ajg

DEPARTAMENTO DE CIENCIAS E INGENIERÍA DE LA COMPUTACIÓN - UNIVERSIDAD NACIONAL DEL SUR - BAHÍA BLANCA - ARGENTINA

Defeasible Logic Programing (DeLP)

DeLP provee un lenguaje de representación de conocimiento que permite representar información tentativa y potencialmente en conflicto [1].



El conocimiento que se tiene sobre un tema, no siempre es completo y definitivo (por ejemplo un virus nuevo que afecta a humanos). El conocimiento puede ser incompleto, pueden haber posturas diferentes, e incluso incertezas propias del tema en cuestión.



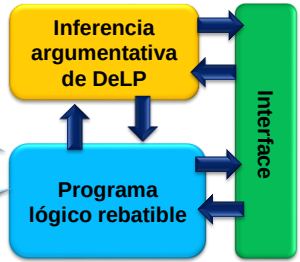
[1] Programación Lógica Rebatible**.** Alejandro J. García. (Capítulo 2) Tesis de Doctor en Ciencias de la Computación. (2000) http://www.cs.uns.edu.ar/~ajg/papers/AGarciaPhD.pdf

Defeasible Logic Programing (DeLP)

DeLP provee un lenguaje de representación de conocimiento que permite representar información tentativa y potencialmente en conflicto [1].

La representación de información tentativa permite expresar reglas generales que se cumplen en la mayoría de los casos, y además, reglas que establecen en que circunstancias pueden darse excepciones.

La representación de información potencialmente en conflicto permite expresar diferentes posturas y opiniones que pueden provenir de diferentes fuentes de información.



El conocimiento que se tiene sobre un tema, no siempre es completo y definitivo (por ejemplo un virus nuevo que afecta a humanos). El conocimiento puede ser incompleto, pueden haber posturas diferentes, e incluso incertezas propias del tema en cuestión.



[1] Programación Lógica Rebatible**.** Alejandro J. García. (Capítulo 2) Tesis de Doctor en Ciencias de la Computación. (2000) http://www.cs.uns.edu.ar/~ajg/papers/AGarciaPhD.pdf

Defeasible Logic Programing (DeLP)

DeLP provee un lenguaje de representación de conocimiento que permite representar información tentativa y potencialmente en conflicto [1].

Ante una consulta, el mecanismo de inferencia de **DeLP** realiza un proceso exhaustivo, de construcción y evaluación de todos aquellos argumentos relacionados con la consulta que se quiere responder (a favor o en contra).

Inferencia argumentativa de DeLP

El conocimiento que se tiene sobre un tema, no siempre es completo y definitivo (por ejemplo un virus nuevo que afecta a humanos). El conocimiento puede ser incompleto, pueden haber posturas diferentes, e incluso incertezas propias del tema en cuestión.



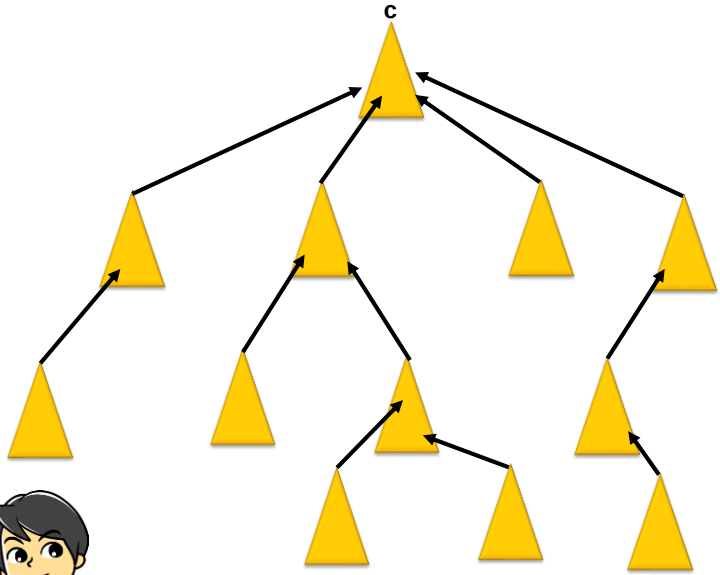
[1] Programación Lógica Rebatible**.** Alejandro J. García. (Capítulo 2) Tesis de Doctor en Ciencias de la Computación. (2000) http://www.cs.uns.edu.ar/~ajg/papers/AGarciaPhD.pdf

Micro-adelanto del proceso de argumentación dialéctica de DeLP

Para computar la repuesta para una consulta **c** DeLP comienza con la construcción de un argumento que sustente a **c**.

Luego, busca todos los posibles derrotadores para ese argumento, y si existen, se construyen los derrotadores de los derrotadores, y así sucesivamente.

Durante este proceso se obtiene un árbol de argumentos, llamado árbol de dialéctica, donde todo nodo (a excepción de la raíz) es un derrotador de su padre.



Árbol de dialéctica marcado

Para determinar si la raíz de un árbol de dialéctica resulta derrotada o no, DeLP realiza un proceso de marcado.

Marcas posibles para un nodo:

D: **Defeated** (derrotado)

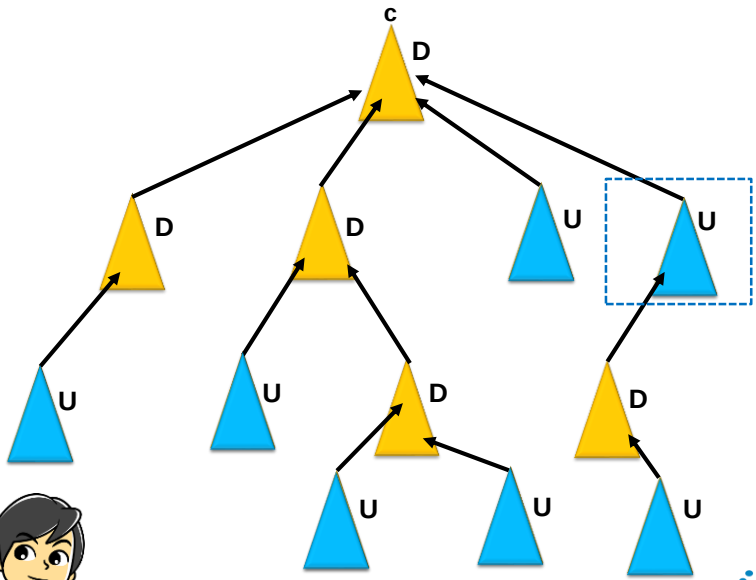
U: **Undefeated** (no derrotado)

Proceso de marcado:

Toda hoja es marcada "**U**"

y todo nodo interno:

- es marcado "**D**" si tiene al menos un hijo marcado "**U**"
- es marcado "**U**" si todos sus hijos están marcados "**D**"



Árbol de dialéctica marcado

Para determinar si la raíz de un árbol de dialéctica resulta derrotada o no, DeLP realiza un proceso de marcado.

Marcas posibles para un nodo:

D: **Defeated** (derrotado)

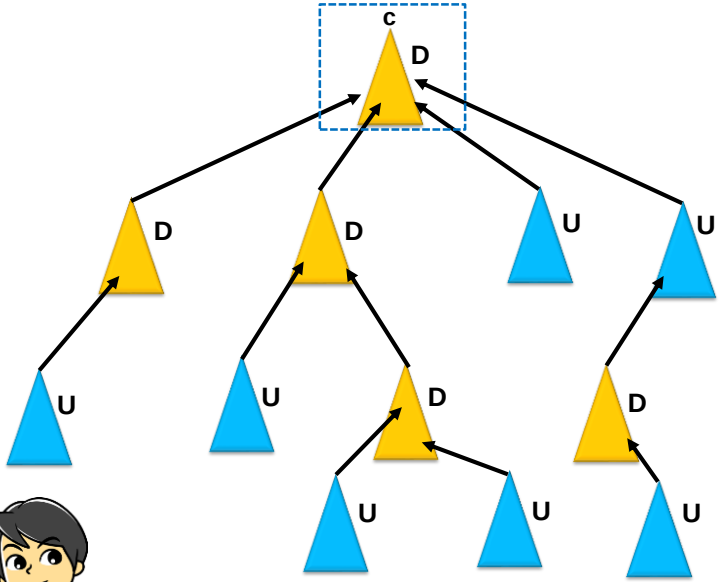
U: **Undefeated** (no derrotado)

Proceso de marcado:

Toda hoja es marcada "**U**"

y todo nodo interno:

- es marcado "**D**" si tiene al menos un hijo marcado "**U**"
- es marcado "**U**" si todos sus hijos están marcados "**D**"

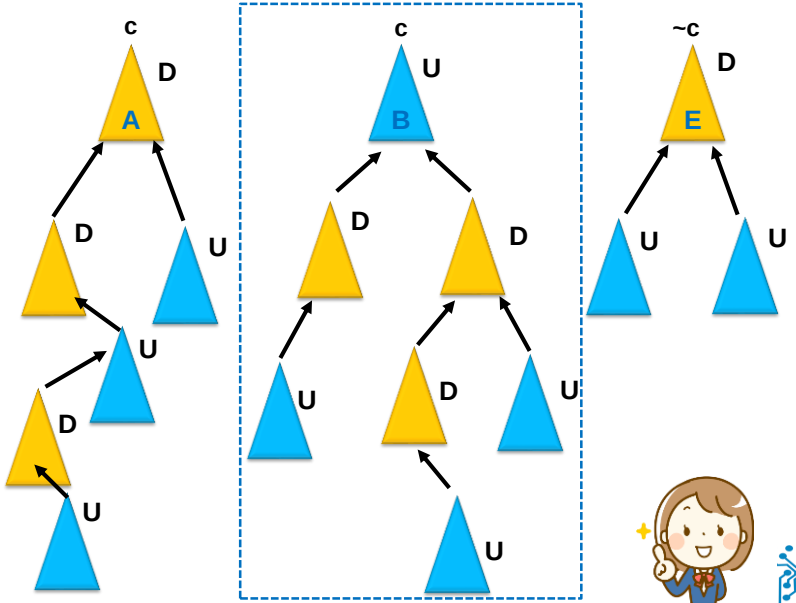


Conclusión garantizada

Ante una consulta **c,** DeLP analizará todos los argumentos que existan para **c** y los que existan para **~c**.

Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

En este ejemplo, la conclusión **c** está garantizada por el argumento **B** que resulta como **U** (undefeted) luego del análisis de todos los argumentos relacionados.

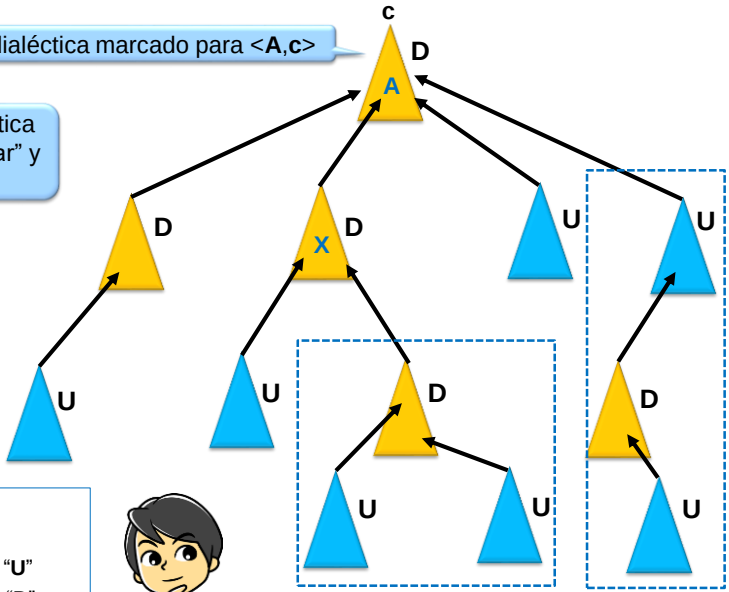


Árbol de dialéctica marcado



Hay partes del árbol de dialéctica marcado que se pueden “podar” y no afectan al resultado.

Árbol de dialéctica marcado para <**A**,**c**>



Marcas posibles para un nodo:

D: **Defeated** (derrotado)

U: **Undefeated** (no derrotado)

Proceso de marcado:

Toda hoja es marcada “**U**” y todo nodo interno:

- es marcado “**D**” si tiene al menos un hijo marcado “**U**”
- es marcado “**U**” si todos sus hijos están marcados “**D**”



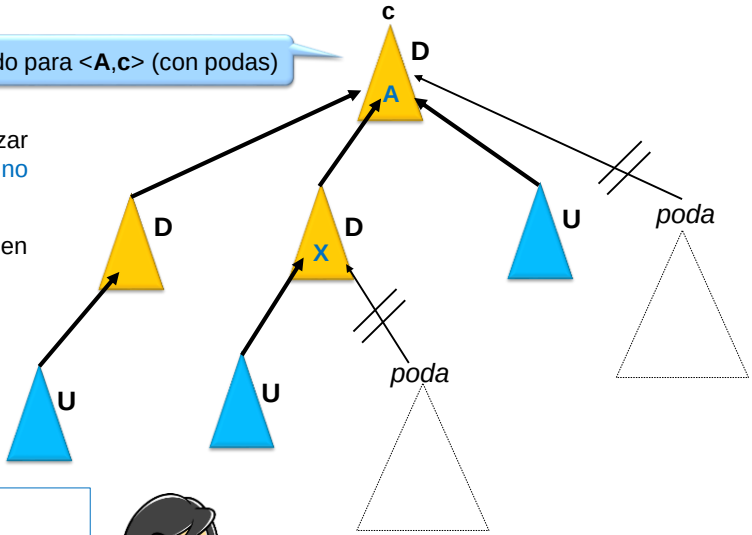
Árbol de dialéctica marcado con poda



Árbol de dialéctica marcado para <**A**,**c**> (con podas)

Durante el proceso de marcado se puede realizar una “poda” y no explorar aquellas líneas que no cambiarán la etiqueta del nodo padre.

La poda que se realice dependerá del orden en que se recorre el árbol.



Marcas posibles para un nodo:

D: **Defeated** (derrotado)

U: **Undefeated** (no derrotado)

Proceso de marcado:

Toda hoja es marcada “**U**” y todo nodo interno:

- es marcado “**D**” si tiene al menos un hijo marcado “**U**”
- es marcado “**U**” si todos sus hijos están marcados “**D**”

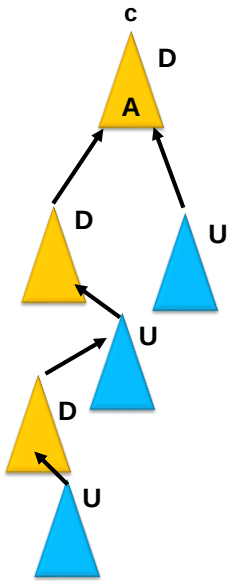


Conclusión garantizada

Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

Ante una consulta **c,** DeLP analizará los argumentos que existan para **c** y los que existan para **~c**.

Veamos paso a paso como es el proceso de determinar una garantía, explorando los árboles de dialéctica en profundidad.



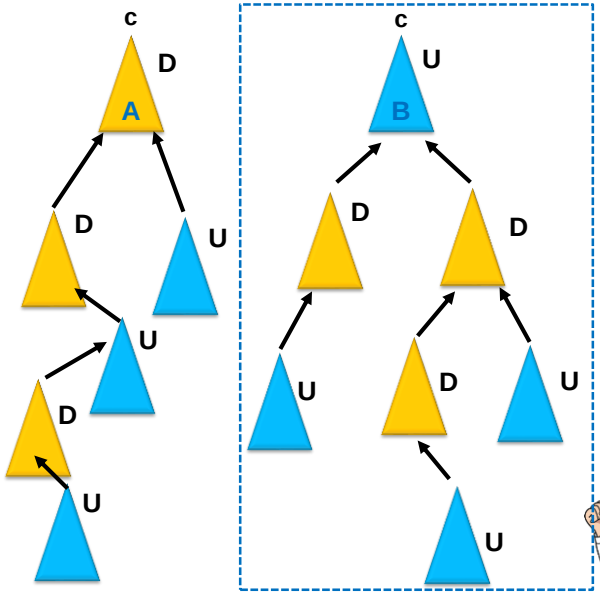




Conclusión garantizada

Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

Ante una consulta **c,** DeLP analizará los argumentos que existan para **c** y los que existan para **~c**.







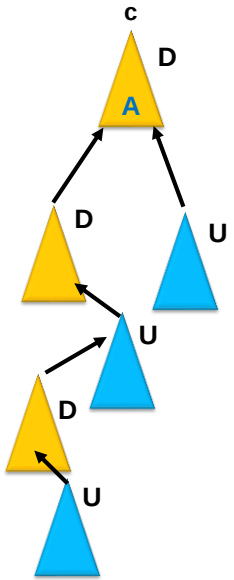
Conclusión garantizada

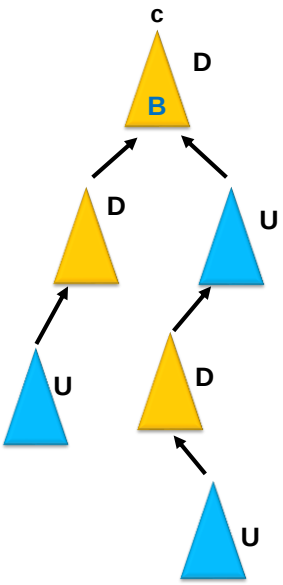
Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

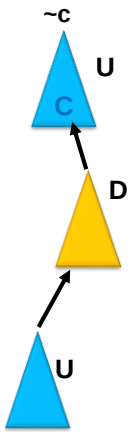
Ante una consulta **c,** DeLP analizará los argumentos que existan para **c** y los que existan para **~c**.



En este ejemplo, hay una garantía para el literal **~c**.









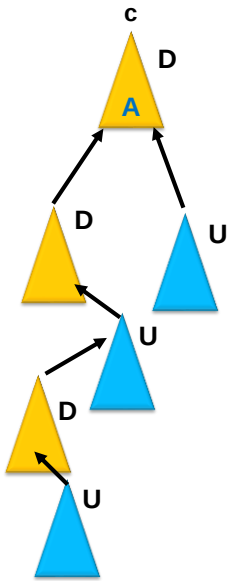
Situación de indecisión

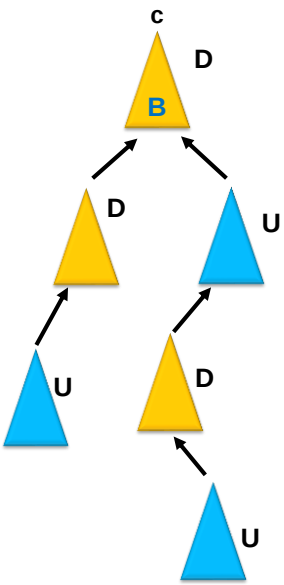
Una conclusión **c** estará garantizada por un programa DeLP, si existe al menos un árbol de dialéctica para un argumento que sustenta **c**, cuya raíz está marcada como **U**.

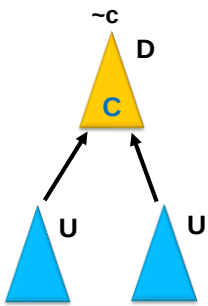
Ante una consulta **c,** DeLP analizará los argumentos que existan para **c** y los que existan para **~c**.

En este ejemplo, no hay una garantía para el literal **c**, y tampoco hay una garantía para el literal **~c**.









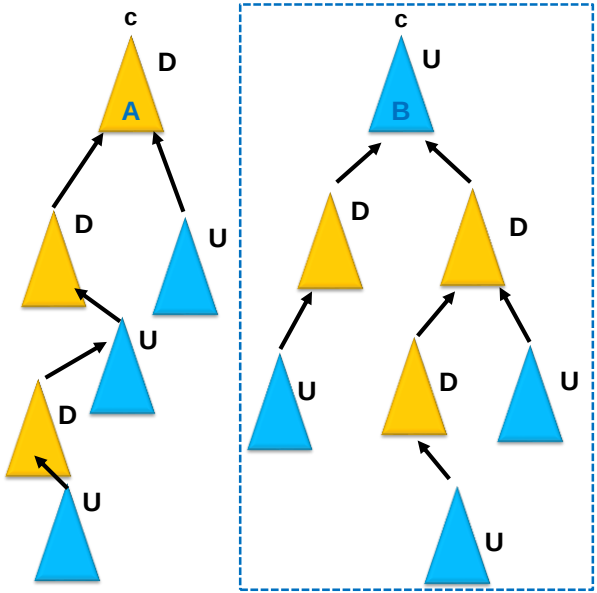


Respuesta a una consulta

La respuesta de un intérprete para una consulta **c** a partir de un PLR **P** es:

yes: cuando **c** está garantizada por **P**.

Los árboles de dialéctica dan, además, una explicación a las respuestas.





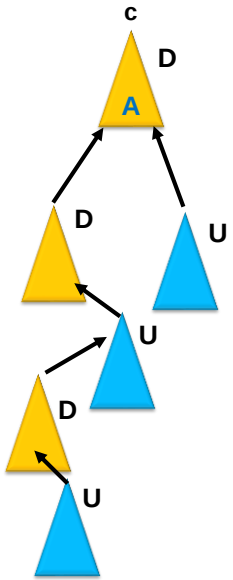
Respuesta a una consulta

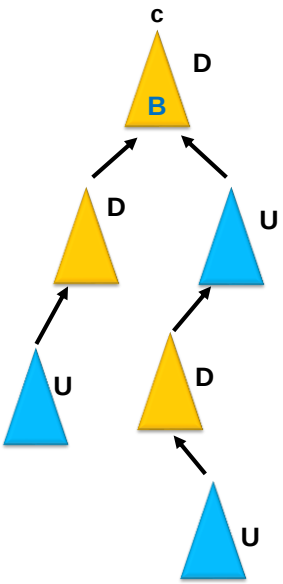
La respuesta de un intérprete para una consulta **c** a partir de un PLR **P** es:

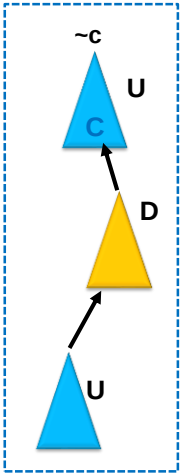
yes: cuando **c** está garantizada por **P**.

no: cuando el *complemento*[1] de **c** está garantizado por **P**.

Los árboles de dialéctica dan, además, una explicación a las respuestas.









[1] El complemento de un literal con respecto a "~" (negación fuerte) se define como: *complemento(c) = ~c* y *complemento(~c) = c*.

Respuesta a una consulta

La respuesta de un intérprete para una consulta **c** a partir de un PLR **P** es:

yes: cuando **c** está garantizada por **P**.

no: cuando el *complemento*[1] de **c** está garantizado por **P**.

unknown: cuando **c** no *pertenece*[2] a **P**.

[1] El complemento de un literal con respecto a "~" (negación fuerte) se define como: *complemento(c) = ~c* y *complemento(~c) = c*.

[2] Un literal *c* pertenece a un programa P cuando *c o complemento(c)* está presente en alguna regla de P.



Respuesta a una consulta

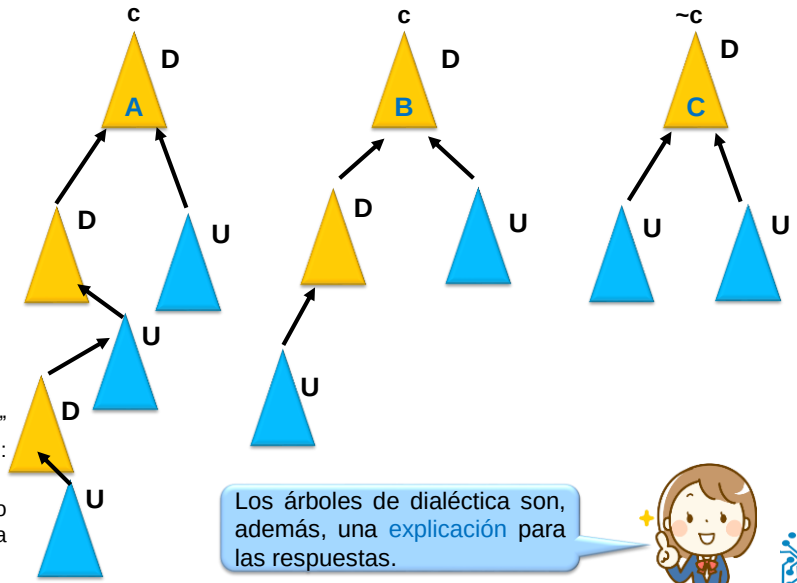
La respuesta de un intérprete para una consulta **c** a partir de un PLR **P** es:

yes: cuando **c** está garantizada por **P**.

no: cuando el *complemento*[1] de **c** está garantizado por **P**.

unknown: cuando **c** no *pertenece*[2] a **P**.

undecided: cuando **c** pertenece a **P**, pero **c** no está garantizada por **P** y su complemento tampoco está garantizado por **P**.



Los árboles de dialéctica son, además, una explicación para las respuestas.

[1] El complemento de un literal con respecto a "~" (negación fuerte) se define como: *complemento(c) = ~c* y *complemento(~c) = c*.

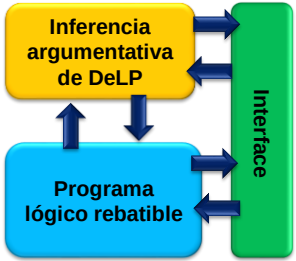
[2] Un literal *c* pertenece a un programa P cuando *c o complemento(c)* está presente en alguna regla de P.

Defeasible Logic Programing (DeLP)

DeLP provee un lenguaje de representación de conocimiento que permite representar información tentativa y potencialmente en conflicto.

Ante una consulta, el mecanismo de inferencia de **DeLP** realiza un proceso exhaustivo, de construcción y evaluación de todos aquellos argumentos relacionados con la consulta que se quiere responder (a favor o en contra).

El proceso de argumentación dialéctica de **DeLP** responderá afirmativamente a una consulta cuando encuentre al menos un argumento a favor que no es derrotado por otros argumentos en contra.



Referencias:

[1] **Programación Lógica Rebatible.** Alejandro J. García. (Capítulo 2) Tesis de Doctor en Ciencias de la Computación. (2000) http://www.cs.uns.edu.ar/~ajg/papers/AGarciaPhD.pdf

[2] **Defeasible logic programming: An argumentative approach.** Alejandro. J. García and Guillermo R. Simari. Journal of Theory and Practice of Logic Programming. (2004) DOI: https://doi.org/10.1017/S1471068403001674

Defeasible Logic Programming

DeLP

- Permite representar información tentativa.
- Las respuestas están sustententadas en un análisis exhaustivo de todos los argumentos relacionados.
- Permite representar información potencialmente en conflicto.
- La preferencia entre argumentos puede adaptarse al dominio de aplicación.
- Las conclusiones están sustentadas con argumentos.
- Es posible dar una explicación detallada de las respuestas.



Sintaxis del lenguaje:

reglas rebatibles, estrictas y hechos

Reglas rebatibles (defeasible rules)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Reglas rebatibles

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Una regla rebatible representa información tentativa y se denota $\mathbf{L_o –< L_1, L_2, \ldots, L_n}$ (con L_i literales)

Un literal es un átomo $p(T_1,...,T_N)$ o un átomo negado $\sim p(T_1,...,T_N)$ donde "~" denota negación fuerte.

Las mayúsculas en los literales denotan variables.
Una regla con variables es un esquema de reglas.

En Δ_m hay dos reglas rebatibles que establecen razones para concluir que una empresa es riesgosa, y una regla que indica razones para concluir que no es riesgosa.

Una regla rebatible (*defeasible rule*) "conclusión –< antecedente" puede leerse así: "si hay razones para creer en el antecedente, entonces hay una razón (rebatible) para creer en la conclusión".



Reglas rebatibles (defeasible rules)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Una regla rebatible representa información tentativa y se denota $L_o -\!< L_1, L_2, \ldots, L_n$ (con L_i literales)

Estas reglas representan información tentativa (o rebatible), porque la conexión que establecen entre el antecedente (las razones) y la conclusión aunque se cumple en muchos casos, podría no cumplirse siempre.
Es decir, pueden haber excepciones.

En Δ_c la primera regla indica que “semáforo en rojo” es una buena razón para concluir “no cruzar”. La segunda regla puede verse como una excepción a la primera, y la tercera regla una excepción a la segunda.

Δ_c {
~cruzar –< semaforo(rojo).
cruzar –< semaforo(rojo), emergencia.
~cruzar –< semaforo(rojo), emergencia, hay_peligro.
}

Puede ser que las tres, sólo dos, sólo una, o ninguna de las reglas puedan derivar conclusiones. Esto dependerá de los hechos percibidos por el agente.



Reglas estrictas (strict rules)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Una regla rebatible representa información tentativa y se denota $L_0 -\!< L_1, L_2, \ldots, L_n$ (con L_i literales)

Una regla estricta representa información segura y se denota $L_0 \leftarrow L_1, L_2, \ldots, L_n$ (con L_i literales)

Aunque sintácticamente la única diferencia es la flecha, hay una enorme diferencia semántica.

A diferencia de las rebatibles, las reglas estrictas representan información segura (no rebatible), porque la conexión que establecen entre el antecedente (las razones) y la conclusión se cumple siempre. Es decir, no pueden haber excepciones.

$\Pi_n =$ { ~abierto ← cerrado. ; abierto ← ~cerrado. ; ~en_casa ← en_supermercado. } Reglas estrictas



Reglas estrictas (strict rules)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).

Reglas estrictas

Una regla rebatible representa información tentativa y se denota L_o –< $L_1, L_2, \ldots, L_n$ (con L_i literales)

Una regla estricta representa información segura y se denota L_o ← $L_1, L_2, \ldots, L_n$ (con L_i literales)

Aunque sintácticamente la única diferencia es la flecha, hay una enorme diferencia semántica.

A diferencia de las rebatibles, las reglas estrictas representan información segura (no rebatible), porque la conexión que establecen entre el antecedente (las razones) y la conclusión se cumple siempre. Es decir, no pueden haber excepciones.



Reglas estrictas (strict rules)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).

Reglas estrictas

Si hay excepciones, entonces no es una regla estricta y de debe representarse como una rebatible.

Una regla rebatible representa información tentativa y se denota L_o –< $L_1, L_2, \ldots, L_n$ (con L_i literales)

Una regla estricta representa información segura y se denota L_o ← $L_1, L_2, \ldots, L_n$ (con L_i literales)

Aunque sintácticamente la única diferencia es la flecha, hay una enorme diferencia semántica.

A diferencia de las rebatibles, las reglas estrictas representan información segura (no rebatible), porque la conexión que establecen entre el antecedente (las razones) y la conclusión se cumple siempre. Es decir, no pueden haber excepciones.

La decisión de representar una regla como estricta o rebatible dependerá del dominio de aplicación del SBC, y es responsabilidad de la persona que codifica el conocimiento.



Hechos (facts)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

A diferencia de las reglas, los hechos no pueden contener variables

Una regla rebatible representa información tentativa y se denota L_o –< $L_1, L_2, \ldots, L_n$ (con L_i literales)

Una regla estricta representa información segura y se denota $L_o \leftarrow L_1, L_2, \ldots, L_n$ (con L_i literales)

Un hecho es un literal fijo (un literal sin variables).

Un literal es un átomo $p(T_1,...,T_N)$ o un átomo negado $\sim p(T_1,...,T_N)$ donde "~" denota negación fuerte.



Hechos (facts)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Una regla rebatible representa información tentativa y se denota L_o –< $L_1, L_2, \ldots, L_n$ (con L_i literales)

Una regla estricta representa información segura y se denota $L_o \leftarrow L_1, L_2, \ldots, L_n$ (con L_i literales)

Un hecho es un literal fijo (un literal sin variables).

Los hechos son una pieza fundamental del conocimiento representado en un programa, ya que deben estar completamente instanciados (no pueden contener variables) y serán los encargados de propagar esas instancias en las derivaciones de las conclusiones del programa.



¿cómo se obtienen los argumentos?

Derivaciones rebatibles (defeasible derivation)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Una derivación rebatible para un literal L a partir de un PLR (Π , Δ) es una secuencia finita de literales fijos $L_1, L_2, \ldots, L_n = L$ tal que para cada literal L_i

- L_i es un hecho en Π, o
- existe en (Π , Δ) una instancia de una regla (estricta o rebatible) con cabeza L_i y cuerpo $B_1, B_2, \ldots, B_k$ donde todo literal del cuerpo está en $L_1, L_2, \ldots, L_{i-1}$



Derivaciones rebatibles (defeasible derivation)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

R1: comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
R3: buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Una derivación rebatible para un literal L a partir de un PLR (Π , Δ) es una secuencia finita de literales fijos $L_1, L_2, \ldots, L_n = L$ tal que para cada literal L_i

- L_i es un hecho en Π, o
- existe en (Π , Δ) una instancia de una regla (estricta o rebatible) con cabeza L_i y cuerpo $B_1, B_2, \ldots, B_k$ donde todo literal del cuerpo está en $L_1, L_2, \ldots, L_{i-1}$

Ejemplo 1: Derivación rebatible para "comprar_acciones(acme)"

L1: buen_precio_web(acme)

L2: buen_precio(acme)

L3: comprar_acciones(acme)

La secuencia L1, L2, L3 es una derivación rebatible para el literal "comprar_acciones(acme)" a partir del PLR (Π_m, Δ_m)

El literal L1 es un hecho, L2 es la cabeza de la regla rebatible R3 instanciada y L3 es la cabeza de la regla rebatible R1 instanciada.



Derivaciones rebatibles (defeasible derivation)

Un programa lógico rebatible (PLR) está formado por un conjunto Δ de reglas rebatibles, y un conjunto Π de reglas estrictas y hechos. Un PLR puede indicarse como el par (Π, Δ) o como el conjunto $\Pi \cup \Delta$.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

E1 ~riesgosa(X) ← fuerte(X).
E2 fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Una derivación rebatible para un literal L a partir de un PLR (Π, Δ) es una secuencia finita de literales fijos $L_1, L_2, \ldots, L_n = L$ tal que para cada literal L_i

- L_i es un hecho en Π, o
- existe en (Π, Δ) una instancia de una regla (estricta o rebatible) con cabeza L_i y cuerpo $B_1, B_2, \ldots, B_k$ donde todo literal del cuerpo está en $L_1, L_2, \ldots, L_{i-1}$

Una derivación estricta: sí sólo usa hechos y reglas estrictas de Π.

Ejemplo 2: derivación estricta para "~riesgosa(strong)"

L1: listado_fuertes_web(strong)

L2: fuerte(strong)

L3: ~riesgosa(strong)

La secuencia L1, L2, L3 es una derivación estricta para el literal "~riesgosa(strong)" a partir del PLR (Π_m, Δ_m)

El literal L1 es un hecho, L2 es la cabeza de la regla estricta E2 instanciada y L3 es la cabeza de la regla estricta E1 instanciada.



Derivaciones para literales complementarios

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

R2
comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).

R5
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).

E3
en_fusion(X,Y) ← fusiones_web(X,Y).

~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Ejemplo 1: La secuencia L1, L2, L3 es una derivación rebatible para el literal "comprar_acciones(acme)" a partir del PLR (Π_m, Δ_m)
L1: buen_precio_web(acme)
L2: buen_precio(acme)
L3: comprar_acciones(acme)

Ejemplo 3: La secuencia L1, L2, L3, L4 es una derivación rebatible para el literal "~comprar_acciones(acme)" a partir del PLR (Π_m, Δ_m)
L1: fusiones_web(acme, strong) [que es un hecho del programa]
L2: en_fusion(acme,strong) [por E3 instanciada]
L3: riesgosa(acme) [por R5 instanciada]
L4: "~comprar_acciones(acme) [por R2 instanciada]

El ejemplo 3 muestra dos cosas importantes:

- una derivación puede usar de los dos tipos de reglas, y
- a partir de un PLR se pueden derivar literales complementarios con respecto a "~" (también llamados literales contradictorios o literales en conflicto porque representan dos conclusiones contradictorias).



Conjunto de reglas contradictorio

Un conjunto S de hechos, reglas estrictas y rebatibles se dice contradictorio si, a partir de S es posible obtener derivaciones rebatibles tanto para un literal p, como su complemento ~p.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Por ejemplo, el conjunto $\Pi_m \cup \Delta_m$ es contradictorio ya que vimos que es posible derivar dos literales complementarios: comprar_acciones(acme) y ~comprar_acciones(acme)

Importante: En todo PLR (Π , Δ) se asume que el conjunto Π no es contradictorio. Esto es, los hechos y las reglas estrictas no pueden inferir conclusiones contradictorias.

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

El conjunto Π_m no es contradictorio.



Conjunto de reglas contradictorio

Un conjunto S de hechos, reglas estrictas y rebatibles se dice contradictorio si, a partir de S es posible obtener derivaciones rebatibles tanto para un literal p, como su complemento ~p.

Ejemplo: PLR merval (Π_m, Δ_m)

Δ_m
- comprar_acciones(X) –< buen_precio(X).
- ~comprar_acciones(X) –< riesgosa (X).
- buen_precio(X) –< buen_precio_web(X).
- ~buen_precio(X) –< en_baja_web(X).
- riesgosa(X) –< en_fusion(X,Y).
- riesgosa(X) –< rumores_deuda_web(X).
- ~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Por ejemplo, el conjunto $\Pi_m \cup \Delta_m$ es contradictorio ya que vimos que es posible derivar dos literales complementarios: comprar_acciones(acme) y ~comprar_acciones(acme)

Importante: En todo PLR (Π , Δ) se asume que el conjunto Π no es contradictorio. Esto es, los hechos y las reglas estrictas no pueden inferir conclusiones contradictorias.

Π_m
- ~riesgosa(X) ← fuerte(X).
- fuerte(X) ← listado_fuertes_web(X).
- con_deuda(X) ← balance_negativo(X).
- en_fusion(X,Y) ← fusiones_web(X,Y).
- ~buen_precio_web(skynet).
- buen_precio_web(acme).
- fusiones_web(acme, strong).
- listado_fuertes_web(strong).
- ~listado_fuertes_web(skynet).

El conjunto Π_m no es contradictorio.

Π_c =
- en_supermercado.
- en_casa.
- ~en_casa ← en_supermercado.

Π_c es un ejemplo de un conjunto que no es válido en DeLP porque es contradictorio



Defeasible Logic Programming

Permite representar información tentativa.

Permite representar información potencialmente en conflicto.

DeLP

Las conclusiones están sustentadas con argumentos.



Argumentos

Argumento



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto **A**$\cup\Pi$

2- (consistencia) el conjunto de reglas **A**$\cup\Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Δ_m

- comprar_acciones(X) –< buen_precio(X).
- ~comprar_acciones(X) –< riesgosa (X).
- buen_precio(X) –< buen_precio_web(X).
- ~buen_precio(X) –< en_baja_web(X).
- riesgosa(X) –< en_fusion(X,Y).
- riesgosa(X) –< rumores_deuda_web(X).
- ~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

- ~riesgosa(X) ← fuerte(X).
- fuerte(X) ← listado_fuertes_web(X).
- con_deuda(X) ← balance_negativo(X).
- en_fusion(X,Y) ← fusiones_web(X,Y).
- ~buen_precio_web(skynet).
- buen_precio_web(acme).
- fusiones_web(acme, strong).
- listado_fuertes_web(strong).
- ~listado_fuertes_web(skynet).

Ejemplo: a partir del PLR **(Π_m , Δ_m)** se puede obtener el argumento <**N,** ~comprar_acciones(acme) >

N =
- ~comprar_acciones(acme) –< riesgosa(acme).
- riesgosa(acme) –< en_fusion(acme,strong).

(1) Hay una derivación a partir de **N**$\cup\Pi_m$

(2) el conjunto **N** $\cup\Pi_m$ no es contradictorio.

(3) no existe subconjunto propio de **N** cumpla (1) y (2)

Argumento



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto $A \cup \Pi$

2- (consistencia) el conjunto de reglas $A \cup \Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Δ_m

- comprar_acciones(X) –< buen_precio(X).
- ~comprar_acciones(X) –< riesgosa (X).
- buen_precio(X) –< buen_precio_web(X).
- ~buen_precio(X) –< en_baja_web(X).
- riesgosa(X) –< en_fusion(X,Y).
- riesgosa(X) –< rumores_deuda_web(X).
- ~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

- ~riesgosa(X) ← fuerte(X).
- fuerte(X) ← listado_fuertes_web(X).
- con_deuda(X) ← balance_negativo(X).
- en_fusion(X,Y) ← fusiones_web(X,Y).
- ~buen_precio_web(skynet).
- buen_precio_web(acme).
- fusiones_web(acme, strong).
- listado_fuertes_web(strong).
- ~listado_fuertes_web(skynet).

Ejemplo: a partir del PLR **(Π_m , Δ_m)** se puede obtener un argumento para la conclusión ~comprar_acciones(acme)

N = { ~comprar_acciones(acme) –< riesgosa(acme).
riesgosa(acme) –< en_fusion(acme,strong). }

Argumento que sustenta ~comprar_acciones(acme)

Derivación rebatible para ~comprar_acciones(acme)

Ejemplo 3: La secuencia L1, L2, L3, L4 es una derivación rebatible para el literal "~comprar_acciones(acme)" a partir del PLR (Π_m, Δ_m)

L1: fusiones_web(acme, strong) [que es un hecho del programa]

L2: en_fusion(acme,strong) [por E3 instanciada]

L3: riesgosa(acme) [por R5 instanciada]

L4: "~comprar_acciones(acme) [por R2 instanciada]

Algunos detalles sobre las condiciones de los argumentos



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto $A \cup \Pi$

2- (consistencia) el conjunto de reglas $A \cup \Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

$$\Pi_n = \left\{ \begin{array}{l} \sim en_casa \leftarrow en_supermercado. \\ en_supermercado. \\ de_noche. \\ festejo. \end{array} \right\}$$

$$\Delta_n = \left\{ \begin{array}{l} en_casa \prec de_noche. \\ preparo_cena \prec de_noche. \\ \sim preparo_cena \prec festejo. \\ acuesto_tarde \prec festejo. \\ en_oficina \prec trabajando. \end{array} \right\}$$

A partir de (Π_n , Δ_n) puedo construir el argumento <**B,** ~preparo_cena>.

$$B = \{ \sim preparo_cena \prec festejo \}$$

Algunos detalles sobre las condiciones de los argumentos



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto $A \cup \Pi$

2- (consistencia) el conjunto de reglas $A \cup \Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Π_n = { ~en_casa ← en_supermercado. ; en_supermercado. ; de_noche. ; festejo. }

Δ_n = { en_casa –< de_noche. ; preparo_cena –< de_noche. ; ~preparo_cena –< festejo. ; acuesto_tarde –< festejo. ; en_oficina –< trabajando. }

A partir de (Π_n , Δ_n) puedo construir el argumento <**B,** ~preparo_cena>.

B = { ~preparo_cena –< festejo }

No todo conjunto de reglas rebatibles es un argumento a partir de (Π_n, Δ_n)

Q = { en_oficina –< trabajando. }

Q no es un argumento para “en_oficina” porque no satisface la condición (1) ya que $Q \cup \Pi_n$ no deriva en “en_oficina”



Algunos detalles sobre las condiciones de los argumentos



Un argumento para una conclusión **h** a partir de un PLR (Π, Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto $A \cup \Pi$

2- (consistencia) el conjunto de reglas $A \cup \Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

$$\Pi_n = \left\{ \begin{array}{l} \sim en_casa \leftarrow en_supermercado. \\ en_supermercado. \\ de_noche. \\ festejo. \end{array} \right\}$$

$$\Delta_n = \left\{ \begin{array}{l} en_casa \prec de_noche. \\ preparo_cena \prec de_noche. \\ \sim preparo_cena \prec festejo. \\ acuesto_tarde \prec festejo. \\ en_oficina \prec trabajando. \end{array} \right\}$$

A partir de (Π_n, Δ_n) puedo construir el argumento <**B,** ~preparo_cena>.

$$B = \{ \sim preparo_cena \prec festejo \}$$

No todo conjunto de reglas rebatibles es un argumento a partir de (Π_n, Δ_n)

$$Q = \{ en_oficina \prec trabajando. \}$$

Q no es un argumento para “en_oficina” porque no satisface la condición (1) ya que $Q \cup \Pi_n$ no deriva en “en_oficina”

$$R = \{ en_casa \prec de_noche. \}$$

R no es un argumento para “en_casa” porque no satisface la condición (2) ya que $R \cup \Pi_n$ es contradictorio



Algunos detalles sobre las condiciones de los argumentos



Un argumento para una conclusión **h** a partir de un PLR (Π, Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto $A \cup \Pi$

2- (consistencia) el conjunto de reglas $A \cup \Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Π_n = { ~en_casa ← en_supermercado. ; en_supermercado. ; de_noche. ; festejo. }

Δ_n = { en_casa –< de_noche. ; preparo_cena –< de_noche. ; ~preparo_cena –< festejo. ; acuesto_tarde –< festejo. ; en_oficina –< trabajando. }

A partir de (Π_n, Δ_n) puedo construir el argumento <**B,** ~preparo_cena>.

B = { ~preparo_cena –< festejo }

No todo conjunto de reglas rebatibles es un argumento a partir de (Π_n, Δ_n)

Q = { en_oficina –< trabajando. }

Q no es un argumento para "en_oficina" porque no satisface la condición (1) ya que $Q \cup \Pi_n$ no deriva en "en_oficina"

R = { en_casa –< de_noche. }

R no es un argumento para "en_casa" porque no satisface la condición (2) ya que $R \cup \Pi_n$ es contradictorio



S = { ~preparo_cena –< festejo ; acuesto_tarde –< festejo. }

S no es un argumento para "~preparo_cena" porque no satisface la condición (3) ya que $B \subset S$

Argumento



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto **A**$\cup\Pi$

2- (consistencia) el conjunto de reglas **A**$\cup\Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Δ_m

- comprar_acciones(X) –< buen_precio(X).
- ~comprar_acciones(X) –< riesgosa (X).
- buen_precio(X) –< buen_precio_web(X).
- ~buen_precio(X) –< en_baja_web(X).
- riesgosa(X) –< en_fusion(X,Y).
- riesgosa(X) –< rumores_deuda_web(X).
- ~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

- ~riesgosa(X) ← fuerte(X).
- fuerte(X) ← listado_fuertes_web(X).
- con_deuda(X) ← balance_negativo(X).
- en_fusion(X,Y) ← fusiones_web(X,Y).
- ~buen_precio_web(skynet).
- buen_precio_web(acme).
- fusiones_web(acme, strong).
- listado_fuertes_web(strong).
- ~listado_fuertes_web(skynet).

Ejemplo: a partir del PLR **(Π_m , Δ_m)** se puede obtener el argumento **<N,** ~comprar_acciones(acme) >

N =

- ~comprar_acciones(acme) –< riesgosa(acme).
- riesgosa(acme) –< en_fusion(acme,strong).

Observaciones importantes:

- Un argumento es un par < conjunto de reglas, conclusión >.
- El conjunto solo incluye reglas rebatibles e instanciadas del PLR.
- La conclusión de un argumento es única y está instanciada.
- El objetivo de que el conjunto de reglas sea mínimo es que no haya reglas de más (todas las reglas son necesarias para la derivación).
- También decimos que el argumento <A,h> sustenta la conclusión h.
- Para una conclusión h puede haber varios argumentos que la sustenten <A_1,h> <A_2,h> <A_3,h>, cada uno representará una razón diferente para h.
- El argumento para un hecho es el conjunto vacío.

Argumento



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto **A**$\cup\Pi$

2- (consistencia) el conjunto de reglas **A**$\cup\Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Algunos argumentos que se pueden obtener a partir del PLR (Π_m , Δ_m)

<**A,** comprar_acciones(acme) >

A = { comprar_acciones(acme) –< buen_precio(acme).
buen_precio(acme) –< buen_precio_web(acme). }

<**N,** ~comprar_acciones(acme) >

N = { ~comprar_acciones(acme) –< riesgosa(acme).
riesgosa(acme) –< en_fusion(acme,strong). }

Argumento



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto $\mathbf{A} \cup \Pi$

2- (consistencia) el conjunto de reglas $\mathbf{A} \cup \Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Algunos argumentos que se pueden obtener a partir del PLR (Π_m , Δ_m)

<**A,** comprar_acciones(acme) >

A = { comprar_acciones(acme) –< buen_precio(acme).
buen_precio(acme) –< buen_precio_web(acme). }

<**N,** ~comprar_acciones(acme) >

N = { ~comprar_acciones(acme) –< riesgosa(acme).
riesgosa(acme) –< en_fusion(acme,strong). }

< **X**, riesgosa(acme) >

X = { riesgosa(acme) –< en_fusion(acme,strong). }

Argumento



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto **A**$\cup\Pi$

2- (consistencia) el conjunto de reglas **A**$\cup\Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Algunos argumentos que se pueden obtener a partir del PLR **(Π_m , Δ_m)**

<**A,** comprar_acciones(acme) >

A = { comprar_acciones(acme) –< buen_precio(acme).
buen_precio(acme) –< buen_precio_web(acme). }

<**N,** ~comprar_acciones(acme) >

N = { ~comprar_acciones(acme) –< riesgosa(acme).
riesgosa(acme) –< en_fusion(acme,strong). }

< **X**, riesgosa(acme) >

X = { riesgosa(acme) –< en_fusion(acme,strong). }

< **R**, ~riesgosa(acme) >

R = { ~riesgosa(acme) –<en_fusion(acme,strong), fuerte(strong). }

Argumento



Un argumento para una conclusión **h** a partir de un PLR (Π , Δ) es un par <**A, h**> donde **h** es un literal fijo y **A** es conjunto de reglas rebatibles instanciadas de Δ que cumple estas tres condiciones:

1- (derivación) existe una derivación rebatible para **h** a partir del conjunto **A**$\cup\Pi$

2- (consistencia) el conjunto de reglas **A**$\cup\Pi$ no es contradictorio

3- (minimalidad) no existe un subconjunto propio de **A** que cumpla con las condiciones (1) y (2)

Δ_m

comprar_acciones(X) –< buen_precio(X).
~comprar_acciones(X) –< riesgosa (X).
buen_precio(X) –< buen_precio_web(X).
~buen_precio(X) –< en_baja_web(X).
riesgosa(X) –< en_fusion(X,Y).
riesgosa(X) –< rumores_deuda_web(X).
~riesgosa(X) –< en_fusion(X,Y), fuerte(Y).

Π_m

~riesgosa(X) ← fuerte(X).
fuerte(X) ← listado_fuertes_web(X).
con_deuda(X) ← balance_negativo(X).
en_fusion(X,Y) ← fusiones_web(X,Y).
~buen_precio_web(skynet).
buen_precio_web(acme).
fusiones_web(acme, strong).
listado_fuertes_web(strong).
~listado_fuertes_web(skynet).

Algunos argumentos que se pueden obtener a partir del PLR **(Π_m , Δ_m)**

<**A,** comprar_acciones(acme) >

A = { comprar_acciones(acme) –< buen_precio(acme).
buen_precio(acme) –< buen_precio_web(acme). }

<**N,** ~comprar_acciones(acme) >

N = { ~comprar_acciones(acme) –< riesgosa(acme).
riesgosa(acme) –< en_fusion(acme,strong). }

< **X**, riesgosa(acme) >

X = { riesgosa(acme) –< en_fusion(acme,strong). }

< **R**, ~riesgosa(acme) >

R = { ~riesgosa(acme) –<en_fusion(acme,strong), fuerte(strong). }

< **G**, ~riesgosa(strong) > **G = { }** (el conjunto vacío)

Hay más detalles técnicos para compartir…



Muchas gracias por la atención!

CONTINUARÁ

